CO1035: Game Concepts
Lab 3A: Finite State Machines

1. Introduction

This lab will introduce you to one of the most important concepts in game development. That topic is Finite State Machines (or FSMs for short). This lab is very important, as in games, everything can be considered an FSM. Try to understand the topic, and if you have any questions, please ask for help from your tutor.

Terminology:

Finite State Machines (FSMs) are a fundamental concept in game development used to control the behaviour of game characters, objects, and systems. An FSM consists of a set of predefined **states**, **transitions**, and **conditions** that determine how an entity behaves and moves between different states. At any given moment, an entity can only be in one state, and transitions occur when specific conditions or events are triggered. For example, an enemy NPC might have states such as Idle, Patrol, Chase, Attack, and Death. The enemy may transition from Patrol to Chase when the player enters its detection range, and from Chase to Attack when the player is close enough. FSMs are widely used in game development for implementing **Artificial Intelligence (AI), character animations, gameplay mechanics, user interfaces, and game logic**. Their main advantages include simplicity, predictability, modularity, and ease of debugging, making them an essential tool for developing structured and responsive game systems.

2. Using Booleans to Store Game State

1. Create a new TL-Engine project named **Lab3A_SphereStates_Project**. Download this lab's media folder from Blackboard, then copy the images inside the zipped folder into your project's folder. To do that, inside your new project, create a folder called Media, and include the path using the code you learned the previous week.
2. Use **Floor.x** to create a floor model at the origin, then create a camera at location (0, 70, -100). You don't need to move the camera at all today, so set the camera type to '**kManual**'.
3. **Rotate** the camera 20 units along the x-axis so that it is facing downwards.
4. Use the **sphere.x** mesh to create a sphere model at (0, 10, 0) so that it is positioned on the floor.

```
IMesh* sphereMesh = myEngine->LoadMesh("Sphere.x"); //declare variable and load mesh
IModel* sphere = sphereMesh->CreateModel(0, 10, 0 ); //declare variable and create model
```

5. Declare a Boolean variable called "**sphereExists**" and set its value to "true". The variable should be declared outside the game loop.
6. The logic of the program we want to code is as follows:

- If the Return key is hit and the **sphereExists** is "**true**", this means that the sphere exists, therefore the sphere model is removed and **sphereExists** is set to "**false**".
- If the Return key is hit and **sphereExists** is "**false**", this means that the sphere does not exist; therefore, the sphere model is created, and **sphereExists** is set to "**true**".

7. It is possible to remove a model using the **'RemoveModel'** method of its mesh. For example, the code below will remove the 'sphere' created earlier:

```
    sphereMesh->RemoveModel( sphere ); //remove model
```

Note that the **'RemoveModel'** method is passed the IModel* variable of the model to be removed as a parameter.

8. The removal of a model means that it is destroyed. It is no longer possible to use the model since it no longer exists and, indeed, if you tried (for example) to move the model after removal, you would generate a real-time error.
9. Although the model has been destroyed, the **IModel*** variable still exists. This is because the **IModel*** variable is a particular kind of variable called a pointer, which is declared using an asterisk (*****). **The pointer still exists, but the removal of the model means it no longer has anything to point at. However, the IModel variable can be reused to point to a newly created model.** For example, the code below serves no real purpose, but it does show the process of removing a model and reusing its **IModel*** variable:

```
IModel* sphere = sphereMesh->CreateModel( ); // declare variable and create model
sphereMesh->RemoveModel( sphere ); // remove model
sphere = sphereMesh->CreateModel( ); // create new model
```

10. Back to the **SphereStates** project, the code to implement the program logic is as follows (it should be added to the game loop):

```
if(myEngine->KeyHit(Key_Return))
{
	if(sphereExists == true)
	{
		sphereMesh->RemoveModel(sphere);
		sphereExists = false;
	}
	else
	{
		sphere = sphereMesh->CreateModel(0, 10, 0);
		sphereExists = true;
	}
}
```

In this fashion, you can track the state of your program. In this particular example, you are keeping track of whether or not the sphere model exists. Furthermore, you are using the state of the sphere to determine what action the program needs to take: whether to create the sphere or to remove it. States are used extensively within game programming and throughout software engineering in general.

3. Using Enums to Store Game State

1. Booleans are useful in handling binary states, e.g., true or false, exists or does not exist, left or right, up or down, etc. For more complex situations, however, enumerations should be used.
2. Enums offer several advantages, including improved readability because they state what they mean, and extensibility since you can easily add a new state as and when required.
3. Let's add the ability to change the color of the sphere (by changing its skin) while using **Enums** to keep track of the color state.
4. To implement this logic, start by creating an Enumeration named **EColourStates** outside of the game loop for storing the possible colors that the sphere can have. For now, they should be Red, Green, and Blue. Then create an **Enum** variable of type **EColourStates** named **sphereColour** and initialize it to Red, as you want the sphere to have a Red skin when the program starts (also set the skin of the sphere model to "**Red.jpg**" when it is created).

```
enum EColourStates {Red, Green, Blue};
EColourStates sphereColour = Red;
```

5. Next, write code to check if the Space key is **hit**; if it is, then update **sphereColour** as specified below:
   - If **sphereColour** is Red, change it to Green, and update the sphere's skin.
   - If **sphereColour** is Green, change it to Blue, and update the sphere's skin.
   - If **sphereColour** is Blue, change it to Red, and update the sphere's skin.

   The code that implements this logic is provided below:

```
if (myEngine->KeyHit(Key_Space)) {
	if (sphereColour == Red) {
		sphereColour = Green;
		sphere->SetSkin("Green.jpg");
	}
	else if (sphereColour == Green){
		sphereColour = Blue;
		sphere->SetSkin("Blue.jpg");
	}
	else if (sphereColour == Blue){
		sphereColour = Red;
		sphere->SetSkin("Red.jpg");
	}
}
```

6. You may have noticed that trying to change the colour of the sphere when it does not exist causes an error. If you haven't, then run the program, press Return so that the sphere disappears, then press Space, you should get a runtime error. To fix this, the program should only change the skin of the sphere if the Space key is hit AND if **sphereExists** is true.

```
if (myEngine->KeyHit(Key_Space) && sphereExists == true) {
if (sphereColour == Red) {
sphereColour = Green;
sphere->SetSkin("Green.jpg");
```

```
}
else if (sphereColour == Green){
sphereColour = Blue;
sphere->SetSkin("Blue.jpg");
}
else if (sphereColour == Blue){
sphereColour = Red;
sphere->SetSkin("Red.jpg");
}
}
```

7. Using the skins provided for this lab, update your program so that when the Space key is hit:
   - If sphereColour is Red, change it to Orange, and update the sphere's skin.
   - If sphereColour is Orange, change it to Yellow, and update the sphere's skin.
   - If sphereColour is Yellow, change it to Green, and update the sphere's skin.
   - If sphereColour is Green, change it to Blue, and update the sphere's skin.
   - If sphereColour is Blue, change it to Indigo, and update the sphere's skin.
   - If sphereColour is Indigo, change it to Violet, and update the sphere's skin.
   - If sphereColour is Violet, change it to Red, and update the sphere's skin.

4. Separating State Transition and Action

1. Let's add the ability to choose a direction for the sphere to rotate towards.
2. Start by creating an Enumeration named **ERotationDirection** outside of the game loop for storing the possible directions of rotation: Upward, Downward, Rightward, Leftward. Then create an Enum variable of type **ERotationDirection** named **sphereRotationDirection** and initialize it to Rightward.

```
enum ERotationDirection {Upward, Downward, Rightward, Leftward};
ERotationDirection sphereRotationDirection = Rightward;
```

3. The logic we want to add to our program is as follows:
   - if the Right Arrow key is hit, change **sphereRotationDirection** to Rightward, keep rotating the sphere rightward until **sphereRotationDirection** is changed
   - if the Left Arrow key is hit, change **sphereRotationDirection** to Leftward, keep rotating the sphere leftward until **sphereRotationDirection** is changed
   - if the Up Arrow key is hit, change **sphereRotationDirection** to Upward, keep rotating the sphere upward until **sphereRotationDirection** is changed
   - if the Down Arrow key is hit, change **sphereRotationDirection** to Downward, keep rotating the sphere downward until **sphereRotationDirection** is changed.

     Note that the action associated with a state transition is not executed once but executed until the state changes again. Therefore, unlike in Sections 2 and 3 where the state transition and action codes were executed inside the same code block, here, you need to

separate the code that handles state transitions and the code that handles the action associated with a given state.

4. First, inside the game loop, write code to check if any of the Arrow Keys is hit, and update **sphereRotationDirection** based on the logic provided above.

```
if (myEngine->KeyHit(Key_Up)) {
sphereRotationDirection = Upward;
}
else if (myEngine->KeyHit(Key_Down)) {
sphereRotationDirection = Downward;
}
else if (myEngine->KeyHit(Key_Left)) {
sphereRotationDirection = Leftward;
}
else if (myEngine->KeyHit(Key_Right)) {
sphereRotationDirection = Rightward;
}
```

5. Next, inside the game loop, write code to check the value of **sphereRotationDirection** and rotate the sphere accordingly.

```
if (sphereRotationDirection == Upward) {
sphere->RotateX(rotationSpeed);
}
else if (sphereRotationDirection == Downward) {
sphere->RotateX(-rotationSpeed);
}
else if (sphereRotationDirection == Leftward) {
sphere->RotateY(-rotationSpeed);
}
else if (sphereRotationDirection == Rightward) {//what is a more efficient way of
checking for this last state?
sphere->RotateY(rotationSpeed);
}
```

Declare **rotationSpeed** as a float outside the game loop and assign an appropriate value to it.

6. Finally, you need to make sure that the sphere is only rotated if it exists to avoid a runtime error. Try doing this yourself.

5. State Transition Diagrams

Draw 3 state transition diagrams to show the three behaviors of the sphere that you have implemented so far (each behavior should have its own separate STD).

6. Extra Exercise - Creating a Simple Platformer

Use what you have learnt so far to create an **endless platformer game about a gravity-defying cube**. You can do this within or outside of the lab session. Try the extra lab activity available on Blackboard.